Module GPS NEO-6M sur Raspberry Pi 5

Lecture UART, décodage NMEA, tests et mise en service

Auteur : Thibault Marcoux

Cours : Informatique embarquée — Master 1, HEH

Projet : Info-Embarque (branche `test/gps`)

Date : 9 novembre 2026

Résumé. Ce rapport décrit l'intégration d'un récepteur GPS u-blox NEO-6M dans le projet Info-Embarque : initialisation de l'UART du Raspberry Pi, décodage des trames NMEA (RMC et GGA), tests unitaires sur PC, programme de test interactif exécuté sur le Raspberry Pi 5, problèmes rencontrés et résultats obtenus. Le module a fourni une position valide en extérieur (5 à 6 satellites, à Mons).

Sommaire

1. Objectif
2. Matériel et câblage
3. Architecture logicielle
4. Le code
5. Tests unitaires (sur PC)
6. Test sur le Raspberry Pi
7. Problèmes rencontrés et solutions
8. Résultats
9. Conclusion et suite du projet
10. Sources

Annexe A — Code source complet

1. Objectif

Le projet Info-Embarque regroupe plusieurs capteurs (températures, gyroscope, impulsions, écran, boutons, enregistrement de données) pilotés par un Raspberry Pi. La partie traitée ici est le **module GPS**, avec trois objectifs :

- ouvrir et configurer correctement le port série (UART) du Raspberry Pi, quel que soit le modèle ;
- lire et décoder les trames NMEA envoyées par le NEO-6M pour obtenir position, altitude, vitesse, cap, heure, date et qualité du signal ;
- valider le code par des tests automatiques sur PC, puis sur le Raspberry Pi avec le vrai module, en pouvant afficher au choix les trames **brutes** ou **traitées**.

2. Matériel et câblage

Élément	Détail
Carte	Raspberry Pi 5 Model B Rev 1.1 — Raspberry Pi OS, noyau 6.18, gcc 14.2
Récepteur GPS	u-blox NEO-6M (carte GY-NEO6MV2), antenne céramique en u.FL
Liaison	UART 9600 bauds, 8 bits, sans parité, 1 bit de stop (8N1), niveaux 3,3 V
Port Linux	`/dev/ttyAMA0` (UART0 du RP1 sur GPIO14/15), activé par `dtparam=uart0=on`

NEO-6M	Raspberry Pi 5	Rôle
VCC	Broche 1 (3,3 V) ou 2 (5 V)	Alimentation (régulateur sur la carte GPS)
GND	Broche 6 (GND)	Masse commune
TX	Broche 10 (GPIO15 / RXD0)	Trames NMEA vers le Pi — indispensable
RX	Broche 8 (GPIO14 / TXD0)	Commandes vers le GPS — facultatif (configuration, diagnostic UBX)

Les liaisons TX/RX se croisent : la sortie de l'un va sur l'entrée de l'autre.

3. Architecture logicielle

Le code est découpé en modules indépendants, chacun dans son dossier :

Module	Fichiers	Rôle
Raspi	raspi.h / raspi.c	Énumération des modèles de Raspberry Pi et choix du fichier de périphérique UART correspondant.
Uart	uart.h / uart.c	Ouverture et configuration du port série avec termios (mode brut, 8N1, non bloquant, vitesse).
GPS	gps.h / gps.c	Assemblage des lignes, vérification du checksum, décodage des trames RMC et GGA dans une structure GpsData.
Tests	test_gps.c	22 tests unitaires sur PC, le port série étant remplacé par un pipe.
Tests	test_gps_raspi.c	Programme interactif sur le Pi : affichage brut, traité ou les deux.

Chemin d'une donnée : le NEO-6M envoie environ 6 trames par seconde → l'UART du Pi les reçoit → `read()` lit les octets disponibles sans bloquer → `gps_Read()` accumule les octets jusqu'à une fin de ligne, vérifie le checksum, découpe les champs et met à jour la structure `GpsData`.

Les trames NMEA sont des lignes de texte de la forme `$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,...*47`. Seules deux trames sont décodées, car elles contiennent toutes les informations utiles :

- **RMC** : statut (A = valide, V = invalide), position, vitesse en nœuds, cap, date ;
- **GGA** : qualité du fix, nombre de satellites, HDOP, altitude.

Les autres trames (GSV, GSA, VTG, GLL) sont ignorées par le décodeur mais restent visibles en mode brut.

4. Le code

Les extraits ci-dessous montrent les parties importantes. Le code complet est en annexe A.

4.1 Initialisation de l'UART

Le port est ouvert en non bloquant, puis configuré avec termios. `cfmakeraw()` désactive tout traitement du texte par Linux (écho, conversion des fins de ligne...), les masques sur `c_cflag` fixent le format 8N1, et `VMIN = VTIME = 0` rend `read()` non bloquant.

```
int fd = open(Raspi_GetUArt_Folder(raspi), O_RDWR | O_NOCTTY | O_NONBLOCK);
struct termios t;
tcgetattr(fd, &t);
cfmakeraw(&t);                                  /* pas d'écho, pas de traitement   */
t.c_cflag &= ~(PARENB | CSTOPB | CSIZE | CRTSCTS);  /* éteint parité, 2 stops, taille  */
t.c_cflag |= CS8 | CLOCAL | CREAD;              /* 8 bits, réception activée       */
t.c_cc[VMIN]  = 0;                              /* read() ne bloque pas            */
t.c_cc[VTIME] = 0;
cfsetispeed(&t, UART_BaudToSpeed(baud));        /* 9600 -> B9600 (voir section 7) */
cfsetospeed(&t, UART_BaudToSpeed(baud));
tcsetattr(fd, TCSANOW, &t);                     /* applique la configuration       */
tcflush(fd, TCIOFLUSH);                         /* vide les anciennes données      */
```

4.2 Vérification du checksum NMEA

Le checksum est le OU exclusif de tous les caractères situés entre `$` et `*`, comparé aux deux chiffres hexadécimaux qui suivent l'étoile. Une trame abîmée par un parasite est ainsi rejetée.

```
static int checksum_ok(const char *ligne)
{
    unsigned char somme = 0;
    const char *p = ligne + 1;              /* on saute le '$' */
    while (*p && *p != '*')
        somme ^= (unsigned char)*p++;
    if (*p != '*') return 0;                /* pas de checksum */
    int h = hex_val(p[1]), l = hex_val(p[2]);
    if (h < 0 || l < 0) return 0;
    return somme == (unsigned char)(h * 16 + l);
}
```

4.3 Découpage des champs et conversion des coordonnées

`strtok()` n'est pas utilisable : il saute les champs vides (`,,`) que le GPS envoie tant qu'il n'a pas de position, ce qui décalerait tous les indices. Une fonction `decouper()` dédiée conserve les champs vides. Les coordonnées NMEA sont au format degrés-minutes (`4807.038` = 48° 07,038') et sont converties en degrés décimaux.

```
/* "ddmm.mmmm" + 'N'/'S'  ->  degrés décimaux */
double v   = atof(val);
int    deg = (int)(v / 100);
double min = v - deg * 100;
double dec = deg + min / 60.0;
if (hemi[0] == 'S' || hemi[0] == 'W') dec = -dec;
```

4.4 Lecture non bloquante et assemblage des lignes

Comme `read()` ne bloque pas, il peut renvoyer une demi-trame. `gps_Read()` garde donc les morceaux dans un tampon `static` jusqu'à recevoir la fin de ligne. Un `$` redémarre toujours une nouvelle trame (resynchronisation), et une ligne trop longue est jetée pour éviter tout débordement.

```
if (ch == '$') {                    /* début de trame : on repart à zéro */
    g_len = 0;
    g_ligne[g_len++] = ch;
} else if (ch == '\r' || ch == '\n') {
```

```
    if (g_len > 0) {                /* fin de trame -> décodage */
        g_ligne[g_len] = '\0';
        nb_trames += traiter_ligne(g_ligne, d);
        g_len = 0;
    }
} else if (g_len > 0) {             /* on n'accumule qu'après un '$' */
    if (g_len < GPS_LIGNE_MAX - 1) g_ligne[g_len++] = ch;
    else                           g_len = 0;     /* trop long : trame corrompue */
}
```

4.5 Programme de test interactif (test_gps_raspi.c)

Le programme surveille en même temps le clavier et l'UART avec `select()`. L'utilisateur choisit l'affichage avec une lettre : **b** (brut), **t** (traité), **d** (les deux), **p** (pause), **h** (aide), **q** (quitter). Le modèle de Raspberry Pi est détecté automatiquement via `/proc/device-tree/model`.

Pour que l'affichage brut et le décodage voient exactement les mêmes octets sans se les « voler », le programme lit lui-même l'UART, affiche les octets si demandé, puis les recopie dans un **pipe interne** ; c'est ce pipe qui est donné à `gps_init()`. Le module GPS n'a donc pas eu besoin d'être modifié.

```
ssize_t n = read(uart, buf, sizeof(buf));
if (n > 0) {
    if (mode == MODE_BRUT || mode == MODE_DEUX)
        afficher_brut(buf, n);              /* lignes NMEA telles quelles  */
    write(p[1], buf, n);                     /* copie vers le module GPS    */
    int nb = gps_Read(&gps);                 /* décodage                    */
    if (nb > 0 && (mode == MODE_TRAITE || mode == MODE_DEUX))
        afficher_traite(&gps, nb);
}
```

5. Tests unitaires (sur PC)

Les tests de `GPS/test_gps.c` ne demandent ni le module ni le Raspberry Pi. Le port série est remplacé par un pipe non bloquant : le test écrit des trames d'un côté, `gps_Read()` les lit de l'autre, exactement comme avec l'UART. Les checksums des trames de test sont calculés automatiquement.

Catégorie	Cas vérifiés
Erreurs d'appel	Lecture avant gps_init, pointeur NULL, descripteur fermé (retour -1), rien à lire (retour 0)
Décodage	Trames RMC/GGA de référence NMEA, trame réaliste à Mons, hémisphères Sud/Ouest, nœuds → km/h, heure et date
Pas de fix	Champs vides au démarrage : fix = 0, l'ancienne position n'est pas écrasée, l'heure est quand même lue
Checksum	Faux, absent, invalide (*4G), en minuscules
Filtrage	GSV, GSA, VTG, GLL, PUBX ignorées ; préfixe GN (multi-constellation) accepté
Découpage	Trame coupée en plusieurs lectures, octet par octet, déchets avant le premier $, $ au milieu d'une trame, fins de ligne \n ou \r seules
Robustesse	Ligne de 400 caractères (pas de débordement), trame trop courte, rafale de 60 trames

Résultat : 22 tests, **72 vérifications, 0 échec**, sur le PC (gcc 15) et sur le Raspberry Pi 5 (gcc 14). Les tests passent aussi avec AddressSanitizer et UndefinedBehaviorSanitizer, qui détectent les accès mémoire invalides.

```
gcc -Wall -Wextra -o test_gps GPS/test_gps.c GPS/gps.c -lm
./test_gps
...
72 verifications, 0 echec(s)
```

6. Test sur le Raspberry Pi

Le code est copié sur le Pi dans `~/Info-Embarque`, puis compilé directement sur la carte :

```
gcc -Wall -Wextra -std=gnu2x -o test_gps_raspi \
    GPS/test_gps_raspi.c GPS/gps.c Uart/uart.c Raspi/raspi.c
./test_gps_raspi           # modèle détecté automatiquement
./test_gps_raspi 5         # ou modèle forcé (valeur de l'énumération RASPI)
```

L'option `-std=gnu2x` est nécessaire car `raspi.h` utilise `enum RASPI : uint8_t`, une syntaxe de la norme C23.

Vérifications faites sur le Pi avant le test :

- `dtparam=uart0=on` présent dans `/boot/firmware/config.txt` ;
- GPIO14 et GPIO15 bien en fonction TXD0/RXD0 (`pinctrl get 14,15`) ;
- la console série et le getty sont sur `ttyAMA10` (connecteur de debug), donc ils ne gênent pas `ttyAMA0` ;
- l'utilisateur fait partie du groupe `dialout` ; gpsd est inactif.

7. Problèmes rencontrés et solutions

7.1 Vitesse de l'UART jamais appliquée

Constat	`uart.c` passait le nombre `9600` à `cfsetispeed()`. Or termios attend une constante (`B9600`). Linux rejette la valeur sans erreur visible (le code de retour n'était pas testé) et le port garde son ancienne vitesse, souvent 115200 bauds : le GPS n'aurait envoyé que du charabia.
Solution	Ajout de `UART_BaudToSpeed()` qui convertit 4800…115200 vers les constantes `B4800`…`B115200`.

7.2 Mauvais port sur Raspberry Pi 5

Constat	`raspi.c` associait le Pi 5 à `/dev/ttyAMA10`. Sur le Pi 5, c'est le petit connecteur de debug à 3 broches, qui porte en plus la console Linux. Les broches GPIO14/15 correspondent à `/dev/ttyAMA0`.
Solution	Le Pi 5 renvoie maintenant `/dev/ttyAMA0` (avec `dtparam=uart0=on`, déjà présent dans config.txt).

7.3 Champs vides des trames NMEA

Constat	Tant qu'il n'a pas de position, le GPS envoie des trames comme `$GPRMC,,V,,,,,,,,,,N*53`. `strtok()` fusionnerait les séparateurs successifs et décalerait les champs.
Solution	Fonction `decouper()` qui conserve les champs vides ; la position n'est mise à jour que si le fix est valide.

7.4 Accès SSH au Raspberry Pi

Constat	Le déploiement automatique du code demandait une connexion sans mot de passe ; aucune clé SSH n'existait sur le PC.
Solution	Création d'une clé (`ssh-keygen -t ed25519`) puis installation sur le Pi (`ssh-copy-id`). Le code est ensuite envoyé et compilé à distance.

7.5 Aucune donnée reçue au premier essai

Constat	Le programme démarrait correctement mais aucun octet n'arrivait, même avec `cat /dev/ttyAMA0`. La configuration du Pi était correcte (broches en mode UART, port libre).
Solution	Problème matériel : le module a été rebranché (TX du GPS sur la broche 10). Les trames sont ensuite arrivées normalement.

7.6 Affichage mélangé en mode « les deux »

Constat	Une ligne [TRAITE] venait s'insérer au milieu d'une trame brute, car les octets bruts étaient affichés au fil de l'eau, morceau par morceau.
Solution	Les octets bruts sont mis en tampon et chaque ligne n'est affichée qu'une fois complète.

7.7 Aucun satellite à l'intérieur

Constat	Le module envoyait `$GPGSV,1,1,00` (0 satellite visible) et aucune heure. Le signal GPS est trop faible pour traverser un bâtiment, et un premier démarrage à froid peut prendre plusieurs minutes. Un premier essai en extérieur n'a d'abord rien donné.
Solution	Test en extérieur avec l'antenne orientée vers le ciel, laissée plus longtemps : position obtenue (voir section 8).

7.8 Pas de réponse aux commandes UBX

Constat	Pour diagnostiquer l'antenne et le bruit radio, des requêtes u-blox (MON-VER, MON-HW) ont été envoyées au module, sans réponse.
Solution	Le fil Pi TX → GPS RX (broche 8) n'est pas branché : il n'est pas nécessaire pour lire la position, mais il l'est pour configurer ou diagnostiquer le module.

7.9 Trames brutes tronquées en extérieur (non résolu)

Constat	Sur la deuxième capture, en mode brut, certaines trames sont coupées et collées entre elles (par exemple la fin d'une GSA suivie d'une RMC). En mode traité, on observe aussi des trous de plusieurs secondes et une seule trame décodée par seconde au lieu de deux.
Solution	Hypothèse principale : un second programme lisait le même port en même temps (une autre instance du test ou un `cat` resté ouvert). Les octets sont alors partagés entre les deux lecteurs. Le Pi n'était plus joignable pour le vérifier ; à contrôler avec `fuser -v /dev/ttyAMA0`.

Remarques supplémentaires sur le reste du projet :

- `main.c` inclut encore l'ancien `GPS/GPS.h` et appelle `GPS_init()` : il doit être mis à jour vers `gps_init(int uart)` pour que le projet complet compile ;
- `uart.h` déclare `UART_IsValidBaudRate()` en `static`, ce qui produit un avertissement dans chaque fichier qui l'inclut ; la déclaration devrait être retirée du header.

8. Résultats

Les deux captures ont été prises le 9 novembre 2026, en extérieur, avec le programme `test_gps_raspi` exécuté sur le Raspberry Pi 5.

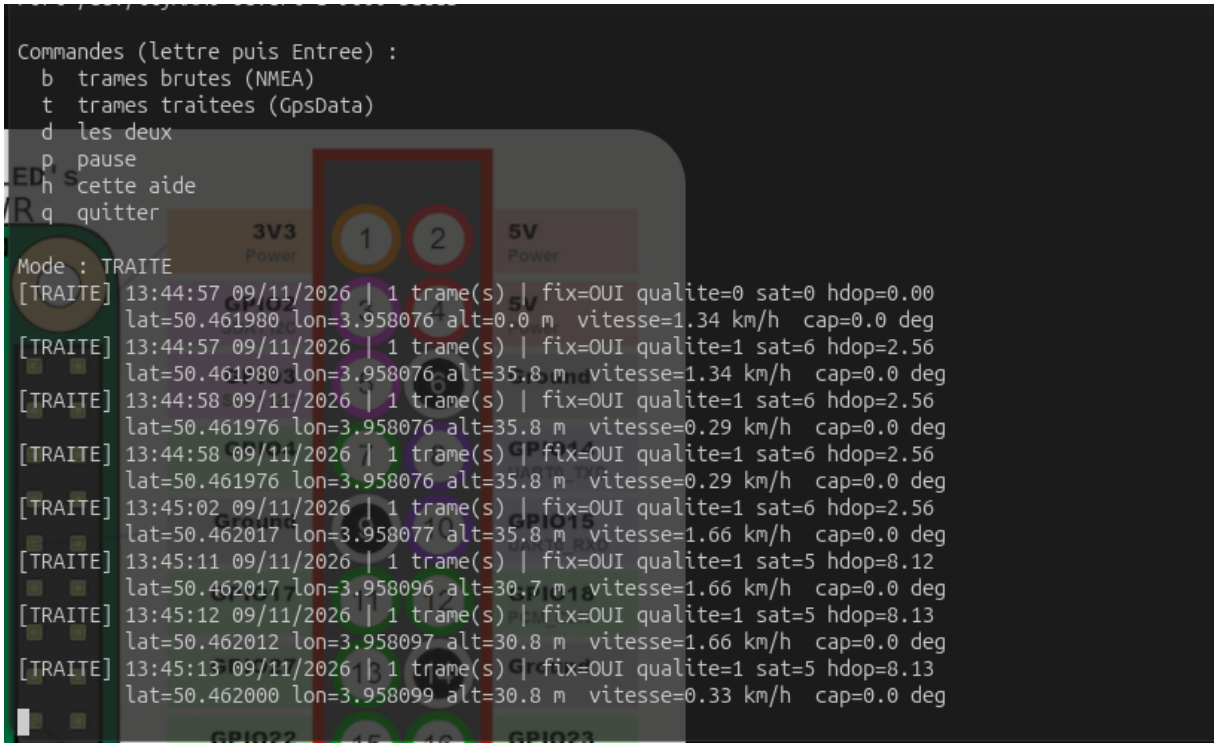


Figure 1 — Mode traité : premier fix à 13:44:57 UTC.

On voit sur la figure 1 :

- **fix=OUI** dès la première trame : la trame RMC valide (statut A) arrive avant la GGA, d'où la première ligne avec `qualite=0 sat=0 alt=0.0` ; la GGA qui suit complète l'information (`qualite=1`, 6 satellites, 35,8 m) ;
- position **50,4620° N, 3,9581° E**, ce qui correspond bien à Mons ;
- l'heure UTC et la date sont correctes (13:44 UTC = 14:44 heure belge, cohérent avec l'horodatage des captures à 14:45).

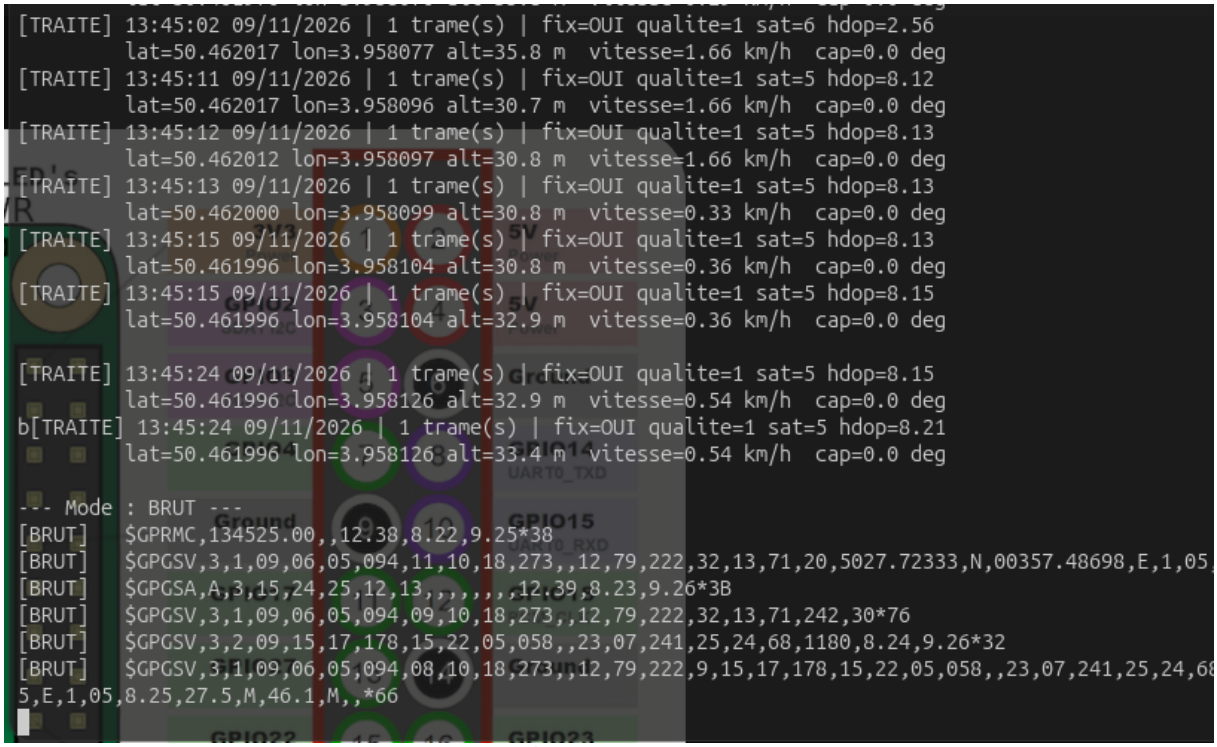


Figure 2 — Suite en mode traité, puis passage en mode brut (commande b).

Analyse des mesures :

Grandeur	Valeurs observées	Interprétation
Satellites	6 puis 5	Suffisant pour un fix 3D (4 minimum)
HDOP	2,56 puis 8,1 à 8,2	2,5 = précision correcte ; au-delà de 5 = précision médiocre, satellites mal répartis dans le ciel
Position	50,46198…50,46202 N / 3,95808…3,95813 E	Variation d'environ 5 m en latitude et 4 m en longitude pour un module immobile
Altitude	30,7 à 35,8 m	Variation de 5 m, normale : l'altitude GPS est moins précise que la position
Vitesse	0,3 à 1,7 km/h	Le module est immobile : c'est le bruit de mesure
Cap	0,0°	Pas de cap calculable sans déplacement

En mode brut (bas de la figure 2), on retrouve les trames u-blox habituelles (RMC, GSV, GSA, GGA) avec la position `5027.72333,N,00357.48698,E`, soit 50° 27,723' N et 3° 57,487' E, identique à la valeur décodée. Certaines lignes sont cependant tronquées (voir problème 7.9).

Conclusion des mesures : la chaîne complète fonctionne, du module GPS jusqu'à la structure `GpsData`. La précision obtenue (quelques mètres, HDOP entre 2,5 et 8) est typique d'un NEO-6M avec sa petite antenne, juste après un démarrage à froid.

9. Conclusion et suite du projet

Le module GPS est opérationnel : l'UART est correctement configurée, les trames NMEA sont lues sans bloquer, vérifiées et décodées, et le code est couvert par des tests automatiques. Le test sur le Raspberry Pi 5 a donné une position valide en extérieur. Deux erreurs du code existant (vitesse de l'UART et port du Pi 5) ont été trouvées et corrigées pendant la mise en service.

Suite à donner :

- vérifier qu'un seul programme lit `/dev/ttyAMA0` et corriger les trames tronquées (problème 7.9) ;
- mettre à jour `main.c` vers la nouvelle interface `gps_init(int uart)` / `gps_Read()` ;
- brancher le fil RX du GPS pour pouvoir configurer le module (cadence, trames utiles) et lire son état (antenne, bruit) avec le protocole UBX ;
- éloigner l'antenne du Raspberry Pi 5 (bruit radio des ports USB 3 et du Wi-Fi) pour améliorer la réception ;
- filtrer la vitesse sous un seuil (par exemple 2 km/h) pour ne pas afficher de mouvement quand le module est immobile.

10. Sources

[1] u-blox, *NEO-6 u-blox 6 GPS Modules — Data Sheet* (GPS.G6-HW-09005). https://www.u-blox.com/en/product/neo-6-series

[2] u-blox, *u-blox 6 Receiver Description including Protocol Specification* (GPS.G6-SW-10018) — trames NMEA et protocole UBX (MON-VER, MON-HW, NAV-STATUS).

[3] NMEA, *NMEA 0183 Standard*. https://www.nmea.org/

[4] Projet gpsd, *NMEA 0183 — Description of the sentences*. https://gpsd.gitlab.io/gpsd/NMEA.html

[5] Raspberry Pi Ltd, *Configuration — UARTs* (alias serial0, mini UART, PL011, Raspberry Pi 5). https://www.raspberrypi.com/documentation/computers/configuration.html

[6] Raspberry Pi Ltd, *RP1 Peripherals* (UART du Raspberry Pi 5). https://datasheets.raspberrypi.com/rp1/rp1-peripherals.pdf

[7] Linux man-pages, *termios(3)*, *select(2)*, *pipe(2)*, *read(2)*. https://man7.org/linux/man-pages/

Annexe A — Code source complet

Fichiers tels qu'ils se trouvent sur la branche `test/gps`. Le fichier de tests unitaires `GPS/test_gps.c` (451 lignes) est décrit en section 5 et n'est pas reproduit ici.

A. GPS/gps.h

```
#ifndef __GPS_H__
#define __GPS_H__

typedef struct {
    int    fix;            /* 1 = position valide (RMC statut 'A'), 0 sinon */
    int    qualite;        /* GGA : 0 = pas de fix, 1 = GPS, 2 = DGPS */
    int    satellites;     /* nombre de satellites utilisés */
    double latitude;       /* degrés décimaux, négatif = Sud */
    double longitude;      /* degrés décimaux, négatif = Ouest */
    double altitude_m;     /* altitude au-dessus du niveau de la mer */
    double vitesse_kmh;
    double cap_deg;        /* cap vrai par rapport au Nord */
    double hdop;           /* précision horizontale, plus petit = meilleur */
    char   heure_utc[9];   /* "hh:mm:ss" */
    char   date[11];       /* "jj/mm/aaaa" */
} GpsData;

/* Associe le module au port série déjà ouvert par uart_ouvrir(). */
void gps_init(int uart);

/* Lit tous les octets disponibles sans bloquer, assemble les lignes
 * et décode les trames complètes dans *d.
 * Retourne le nombre de trames valides décodées (0 si rien de nouveau),
 * ou -1 en cas d'erreur de lecture. */
int gps_Read(GpsData *d);

#endif
```

A. GPS/gps.c

```
#include "gps.h"

#include <ctype.h>
#include <errno.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define GPS_LIGNE_MAX 128      /* une trame NMEA fait au maximum 82 caractères */
#define GPS_CHAMPS_MAX 20

static int  g_uart = -1;
static char g_ligne[GPS_LIGNE_MAX];
static int  g_len  = 0;

void gps_init(int uart)
{
    g_uart = uart;
    g_len  = 0;
}

/* ---------- Outils ---------- */

static int hex_val(char c)
{
    if (c >= '0' && c <= '9') return c - '0';
    c = (char)toupper((unsigned char)c);
    if (c >= 'A' && c <= 'F') return c - 'A' + 10;
    return -1;
}

/* Checksum NMEA = XOR de tous les caractères entre '$' et '*' */
static int checksum_ok(const char *ligne)
{
    unsigned char somme = 0;
    const char *p = ligne + 1;             /* on saute le '$' */
```

```
    while (*p && *p != '*')
        somme ^= (unsigned char)*p++;

    if (*p != '*') return 0;                /* pas de checksum */
    int h = hex_val(p[1]), l = hex_val(p[2]);
    if (h < 0 || l < 0) return 0;
    return somme == (unsigned char)(h * 16 + l);
}

/* Découpe la ligne sur les ',' (garde les champs vides, contrairement à strtok) */
static int decouper(char *s, char *champs[], int max)
{
    int n = 0;
    champs[n++] = s;
    for (; *s; s++) {
        if (*s == '*') { *s = '\0'; break; }
        if (*s == ',') {
            *s = '\0';
            if (n < max) champs[n++] = s + 1;
        }
    }
    return n;
}

/* "ddmm.mmmm" + 'N'/'S'  ->  degrés décimaux. Retourne 0 si champ vide. */
static int convertir_coord(const char *val, const char *hemi, double *res)
{
    if (val[0] == '\0' || hemi[0] == '\0') return 0;
    double v    = atof(val);
    int    deg  = (int)(v / 100);
    double min  = v - deg * 100;
    double dec  = deg + min / 60.0;
    if (hemi[0] == 'S' || hemi[0] == 'W') dec = -dec;
    *res = dec;
    return 1;
}

/* "hhmmss.ss" -> "hh:mm:ss" */
static void copier_heure(const char *src, char *dst)
{
    if (strlen(src) < 6) return;
    dst[0] = src[0]; dst[1] = src[1]; dst[2] = ':';
    dst[3] = src[2]; dst[4] = src[3]; dst[5] = ':';
    dst[6] = src[4]; dst[7] = src[5]; dst[8] = '\0';
}

/* "ddmmyy" -> "jj/mm/20aa" */
static void copier_date(const char *src, char *dst)
{
    if (strlen(src) < 6) return;
    dst[0] = src[0]; dst[1] = src[1]; dst[2] = '/';
    dst[3] = src[2]; dst[4] = src[3]; dst[5] = '/';
    dst[6] = '2';    dst[7] = '0';
    dst[8] = src[4]; dst[9] = src[5]; dst[10] = '\0';
}

/* ---------- Décodage des trames ---------- */

/* $GPRMC,heure,statut,lat,N/S,lon,E/W,vitesse_noeuds,cap,date,...  */
static int decoder_rmc(char *c[], int n, GpsData *d)
{
    if (n < 10) return 0;

    copier_heure(c[1], d->heure_utc);
    copier_date(c[9], d->date);

    d->fix = (c[2][0] == 'A');
    if (!d->fix) return 1;                  /* trame valide, mais pas de position */

    convertir_coord(c[3], c[4], &d->latitude);
    convertir_coord(c[5], c[6], &d->longitude);
    if (c[7][0]) d->vitesse_kmh = atof(c[7]) * 1.852;   /* noeuds -> km/h */
    if (c[8][0]) d->cap_deg     = atof(c[8]);
    return 1;
}

/* $GPGGA,heure,lat,N/S,lon,E/W,qualite,nb_sat,hdop,altitude,M,...  */
```

```
static int decoder_gga(char *c[], int n, GpsData *d)
{
    if (n < 10) return 0;

    copier_heure(c[1], d->heure_utc);
    d->qualite    = atoi(c[6]);
    d->satellites = atoi(c[7]);
    if (c[8][0]) d->hdop = atof(c[8]);

    if (d->qualite > 0) {
        convertir_coord(c[2], c[3], &d->latitude);
        convertir_coord(c[4], c[5], &d->longitude);
        if (c[9][0]) d->altitude_m = atof(c[9]);
    }
    return 1;
}

static int traiter_ligne(char *ligne, GpsData *d)
{
    if (ligne[0] != '$' || !checksum_ok(ligne)) return 0;

    char *champs[GPS_CHAMPS_MAX];
    int n = decouper(ligne + 1, champs, GPS_CHAMPS_MAX);

    /* champs[0] = "GPRMC" : on ignore les 2 lettres de l'émetteur (GP, GN...) */
    if (strlen(champs[0]) != 5) return 0;
    const char *type = champs[0] + 2;

    if (strcmp(type, "RMC") == 0) return decoder_rmc(champs, n, d);
    if (strcmp(type, "GGA") == 0) return decoder_gga(champs, n, d);
    return 0;                                /* GSV, GSA, VTG... ignorées */
}

/* ---------- Lecture ---------- */

int gps_Read(GpsData *d)
{
    if (g_uart < 0 || d == NULL) return -1;

    char buf[256];
    int  nb_trames = 0;

    for (;;) {
        ssize_t n = read(g_uart, buf, sizeof(buf));
        if (n < 0) {
            if (errno == EAGAIN || errno == EWOULDBLOCK || errno == EINTR) break;
            return -1;
        }
        if (n == 0) break;                   /* plus rien à lire pour l'instant */

        for (ssize_t i = 0; i < n; i++) {
            char ch = buf[i];

            if (ch == '$') {                 /* début de trame : on repart à zéro */
                g_len = 0;
                g_ligne[g_len++] = ch;
            } else if (ch == '\r' || ch == '\n') {
                if (g_len > 0) {             /* fin de trame */
                    g_ligne[g_len] = '\0';
                    nb_trames += traiter_ligne(g_ligne, d);
                    g_len = 0;
                }
            } else if (g_len > 0) {          /* on n'accumule qu'après un '$' */
                if (g_len < GPS_LIGNE_MAX - 1)
                    g_ligne[g_len++] = ch;
                else
                    g_len = 0;               /* trop long : trame corrompue */
            }
        }
    }
    return nb_trames;
}
```

A. Uart/uart.h

```
#ifndef __UART_H__
```

```
#define __UART_H__

#include <stdint.h>
#include <stdbool.h>

#include "../Raspi/raspi.h"

#define UART_ERR_BAUDRATE_INVALID -1
#define UART_ERR_RASPI_INVALID -2
#define UART_ERR_IMPOSSIBLE_TO_READ_UART -3
#define UART_ERR_IMPOSSIBLE_TO_INIT_STRUCT -4
#define UART_ERR_IMPOSSIBLE_TO_PARAM_UART -5

int32_t UART_Init(RASPI raspi, uint32_t baud);
void UART_Close(int32_t uart);

static bool UART_IsValidBaudRate(uint32_t baud);

#endif
```

A. Uart/uart.c

```
#include "uart.h"

#include <unistd.h>
#include <termios.h>
#include <fcntl.h>

/* termios attend une constante (B9600...), pas le nombre 9600 */
static speed_t UART_BaudToSpeed(uint32_t baud)
{
    switch (baud) {
        case 4800:   return B4800;
        case 9600:   return B9600;
        case 19200:  return B19200;
        case 38400:  return B38400;
        case 57600:  return B57600;
        case 115200: return B115200;
        default:     return B0;
    }
}

int32_t UART_Init(RASPI raspi, uint32_t baud)
{
    if(!UART_IsValidBaudRate(baud))
        return UART_ERR_BAUDRATE_INVALID;

    if(!Rapsi_IsValid(raspi))
        return UART_ERR_RASPI_INVALID;

    int fd = open(Raspi_GetUArt_Folder(raspi), O_RDWR | O_NOCTTY | O_NONBLOCK);
    if (fd < 0)
        return UART_ERR_IMPOSSIBLE_TO_READ_UART;


    struct termios t;
    if (tcgetattr(fd, &t) < 0) {
        close(fd);
        return UART_ERR_IMPOSSIBLE_TO_INIT_STRUCT;
    }

    cfmakeraw(&t);                  /* pas d'écho, pas de traitement des caractères */
    t.c_cflag &= ~(PARENB | CSTOPB | CSIZE | CRTSCTS);  // étient tous les interrupteur
    t.c_cflag |= CS8 | CLOCAL | CREAD;   /* 8 bits, sans parité, 1 bit de stop */
    t.c_cc[VMIN]  = 0;
    t.c_cc[VTIME] = 0;
    cfsetispeed(&t, UART_BaudToSpeed(baud));   // vitesse d'écoute
    cfsetospeed(&t, UART_BaudToSpeed(baud));   // vitesse d'écriture

    if (tcsetattr(fd, TCSANOW, &t) < 0) {
        close(fd);
        return UART_ERR_IMPOSSIBLE_TO_PARAM_UART;
    }

    tcflush(fd, TCIOFLUSH);         /* vide les anciennes données */
    return fd;
```

```
}


void UART_Close(int32_t uart)
{
    if(uart >= 0)
        close(uart);
}


bool UART_IsValidBaudRate(uint32_t baud)
{
    switch (baud) {
        case 4800:
        case 9600:
        case 19200:
        case 38400:
        case 57600:
        case 115200:
            return 1;

        default:
            return 0;
    }
}
```

A. Raspi/raspi.h

```
#ifndef __RASPI_H__
#define __RASPI_H__

#include <stdint.h>
#include <stdbool.h>

typedef enum RASPI : uint8_t
{
    RASPI_1         = 1,
    RASPI_2         = 2,
    RASPI_3         = 3,
    RASPI_4         = 4,
    RASPI_5         = 5,

    RASPI_ZERO_1    = 0,
    RASPI_ZERO_1W   = 6,
    RASPI_ZERO_2W   = 7,

    RASPI_400       = 8

} RASPI;

bool Rapsi_IsValid(RASPI raspi);

const char* Raspi_GetUArt_Folder(RASPI raspi);

#endif
```

A. Raspi/raspi.c

```
#include "raspi.h"

static const char* UART_FOLDER[] = {"/dev/ttyAMA0",
                                    "/dev/ttyS0",
                                    "/dev/ttyAMA10"
                                };


bool Rapsi_IsValid(RASPI raspi)
{
    switch ((uint8_t)raspi)
    {
        case RASPI_ZERO_1:
        case RASPI_1:
        case RASPI_2:
        case RASPI_ZERO_1W:
        case RASPI_ZERO_2W:
        case RASPI_3:
```

```
        case RASPI_4:
        case RASPI_400:
        case RASPI_5:
            return 1;

        default:
            return 0;
    };
}


const char* Raspi_GetUArt_Folder(RASPI raspi)
{
    switch ((uint8_t)raspi)
    {
        case RASPI_ZERO_1:
        case RASPI_1:
        case RASPI_2:
            return UART_FOLDER[0];

        case RASPI_ZERO_1W:
        case RASPI_ZERO_2W:
        case RASPI_3:
        case RASPI_4:
        case RASPI_400:
            return UART_FOLDER[1];

        case RASPI_5:
            /* GPIO14/15 = ttyAMA0 (avec dtparam=uart0=on).
             * ttyAMA10 est le petit connecteur de debug, réservé à la console. */
            return UART_FOLDER[0];

        default:
            return "";
    };
}
```

A. GPS/test_gps_raspi.c

```
/*
 * Test du GPS NEO-6M sur le Raspberry Pi, avec le vrai module branché.
 *
 * Le programme lit l'UART en continu et affiche, selon la commande tapée :
 *     b + Entrée  ->  trames BRUTES (le texte NMEA tel qu'il sort du GPS)
 *     t + Entrée  ->  trames TRAITÉES (GpsData décodé par gps_Read)
 *     d + Entrée  ->  les DEUX
 *     p + Entrée  ->  pause (les données continuent d'être lues mais pas affichées)
 *     h + Entrée  ->  aide
 *     q + Entrée  ->  quitter
 *
 * Comment ça marche : le programme lit lui-même l'UART, affiche les octets
 * bruts si demandé, puis les recopie dans un pipe interne. C'est ce pipe qui
 * est donné à gps_init() : le module GPS décode donc exactement les mêmes
 * octets que ceux affichés en brut, sans que les deux se volent les données.
 *
 * Compilation (sur le Pi, depuis la racine du projet) :
 *     gcc -Wall -Wextra -std=gnu2x -I Raspi -I Uart -o test_gps_raspi \
 *         GPS/test_gps_raspi.c GPS/gps.c Uart/uart.c Raspi/raspi.c
 *
 * Lancement :
 *     ./test_gps_raspi          modèle de Pi détecté automatiquement
 *     ./test_gps_raspi 4        forcer le modèle (valeur de l'enum RASPI)
 */

#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/select.h>
#include <time.h>
#include <unistd.h>

#include "gps.h"
#include "../Raspi/raspi.h"
#include "../Uart/uart.h"
```

```
#define GPS_BAUD          9600   /* vitesse d'usine du NEO-6M */
#define SILENCE_ALERTE_S  3      /* alerte si rien reçu pendant ce temps */

typedef enum { MODE_BRUT, MODE_TRAITE, MODE_DEUX, MODE_PAUSE } Mode;

static const char *nom_mode(Mode m)
{
    switch (m) {
        case MODE_BRUT:   return "BRUT";
        case MODE_TRAITE: return "TRAITE";
        case MODE_DEUX:   return "BRUT + TRAITE";
        default:          return "PAUSE";
    }
}

static void afficher_aide(void)
{
    printf("\nCommandes (lettre puis Entree) :\n"
           "  b  trames brutes (NMEA)\n"
           "  t  trames traitees (GpsData)\n"
           "  d  les deux\n"
           "  p  pause\n"
           "  h  cette aide\n"
           "  q  quitter\n\n");
}

/* Lit /proc/device-tree/model pour deviner le modèle de Raspberry Pi */
static int detecter_raspi(RASPI *r)
{
    char modele[128] = {0};
    FILE *f = fopen("/proc/device-tree/model", "r");
    if (!f) return 0;
    size_t n = fread(modele, 1, sizeof(modele) - 1, f);
    fclose(f);
    modele[n] = '\0';

    printf("Carte detectee : %s\n", modele);

    /* Les tests les plus précis d'abord ("Zero 2 W" contient aussi "Zero") */
    if      (strstr(modele, "Zero 2 W"))         *r = RASPI_ZERO_2W;
    else if (strstr(modele, "Zero W"))           *r = RASPI_ZERO_1W;
    else if (strstr(modele, "Zero"))             *r = RASPI_ZERO_1;
    else if (strstr(modele, "Raspberry Pi 400")) *r = RASPI_400;
    else if (strstr(modele, "Raspberry Pi 5"))   *r = RASPI_5;
    else if (strstr(modele, "Raspberry Pi 4"))   *r = RASPI_4;
    else if (strstr(modele, "Raspberry Pi 3"))   *r = RASPI_3;
    else if (strstr(modele, "Raspberry Pi 2"))   *r = RASPI_2;
    else if (strstr(modele, "Raspberry Pi"))     *r = RASPI_1;
    else return 0;
    return 1;
}

static const char *texte_erreur_uart(int code)
{
    switch (code) {
        case UART_ERR_BAUDRATE_INVALID:          return "vitesse invalide";
        case UART_ERR_RASPI_INVALID:             return "modele de Raspberry Pi invalide";
        case UART_ERR_IMPOSSIBLE_TO_READ_UART:   return "impossible d'ouvrir le port "
                                                        "(UART active ? groupe dialout ?)";
        case UART_ERR_IMPOSSIBLE_TO_INIT_STRUCT: return "tcgetattr a echoue";
        case UART_ERR_IMPOSSIBLE_TO_PARAM_UART:  return "tcsetattr a echoue";
        default:                                 return "erreur inconnue";
    }
}

/* Affiche les octets bruts ligne par ligne : on garde les morceaux jusqu'au
 * '\n' pour qu'une ligne [TRAITE] ne vienne jamais se glisser au milieu.
 * Texte NMEA tel quel, octets bizarres affichés en <xx>. */
static void afficher_brut(const char *buf, ssize_t n)
{
    static char ligne[512];
    static size_t len = 0;

    for (ssize_t i = 0; i < n; i++) {
        unsigned char c = (unsigned char)buf[i];
        if (c == '\r') continue;
```

```
        if (c == '\n') {
            if (len > 0) {
                ligne[len] = '\0';
                printf("[BRUT]   %s\n", ligne);
                len = 0;
            }
        } else if (len < sizeof(ligne) - 5) {
            if (c >= 32 && c < 127)
                ligne[len++] = (char)c;
            else
                len += (size_t)snprintf(ligne + len, 5, "<%02X>", c);
        }
    }
    fflush(stdout);
}

static void afficher_traite(const GpsData *d, int nb_trames)
{
    printf("[TRAITE] %s %s | %d trame(s) | fix=%s qualite=%d sat=%d hdop=%.2f\n",
           d->heure_utc[0] ? d->heure_utc : "--:--:--",
           d->date[0]      ? d->date      : "--/--/----",
           nb_trames,
           d->fix ? "OUI" : "non",
           d->qualite, d->satellites, d->hdop);

    if (d->fix)
        printf("         lat=%.6f lon=%.6f alt=%.1f m  vitesse=%.2f km/h  cap=%.1f deg\n",
               d->latitude, d->longitude, d->altitude_m, d->vitesse_kmh, d->cap_deg);
    else
        printf("         pas encore de position (mettre l'antenne face au ciel)\n");
    fflush(stdout);
}

int main(int argc, char *argv[])
{
    RASPI raspi;

    if (argc > 1) {
        raspi = (RASPI)atoi(argv[1]);
    } else if (!detecter_raspi(&raspi)) {
        fprintf(stderr, "Modele non reconnu, passe-le en argument (ex : %s 4)\n", argv[0]);
        return 1;
    }

    if (!Rapsi_IsValid(raspi)) {
        fprintf(stderr, "Modele %d invalide\n", (int)raspi);
        return 1;
    }

    const char *port = Raspi_GetUArt_Folder(raspi);
    int uart = UART_Init(raspi, GPS_BAUD);
    if (uart < 0) {
        fprintf(stderr, "UART_Init(%s, %d) : %s\n", port, GPS_BAUD, texte_erreur_uart(uart));
        return 1;
    }
    printf("Port %s ouvert a %d bauds\n", port, GPS_BAUD);

    /* Pipe interne : nous écrivons dedans, le module GPS lit dedans */
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); UART_Close(uart); return 1; }
    fcntl(p[0], F_SETFL, fcntl(p[0], F_GETFL) | O_NONBLOCK);
    gps_init(p[0]);

    GpsData gps = {0};
    Mode mode = MODE_TRAITE;
    time_t derniere_donnee = time(NULL);
    int alerte_affichee = 0;

    afficher_aide();
    printf("Mode : %s\n", nom_mode(mode));

    for (;;) {
        fd_set lecture;
        FD_ZERO(&lecture);
        FD_SET(STDIN_FILENO, &lecture);
        FD_SET(uart, &lecture);
```

```
        struct timeval attente = { .tv_sec = 1, .tv_usec = 0 };
        int max_fd = uart > STDIN_FILENO ? uart : STDIN_FILENO;

        if (select(max_fd + 1, &lecture, NULL, NULL, &attente) < 0) {
            if (errno == EINTR) continue;
            perror("select");
            break;
        }

        /* ---- Commande de l'utilisateur ---- */
        if (FD_ISSET(STDIN_FILENO, &lecture)) {
            char cmd[32];
            if (!fgets(cmd, sizeof(cmd), stdin)) break;     /* Ctrl+D */

            Mode ancien = mode;
            switch (cmd[0]) {
                case 'b': mode = MODE_BRUT;   break;
                case 't': mode = MODE_TRAITE; break;
                case 'd': mode = MODE_DEUX;   break;
                case 'p': mode = MODE_PAUSE;  break;
                case 'h': afficher_aide();    break;
                case 'q': goto fin;
                case '\n': break;
                default:  printf("Commande inconnue, tape h pour l'aide\n"); break;
            }
            if (mode != ancien) printf("--- Mode : %s ---\n", nom_mode(mode));
        }

        /* ---- Données du GPS ---- */
        if (FD_ISSET(uart, &lecture)) {
            char buf[256];
            ssize_t n = read(uart, buf, sizeof(buf));
            if (n < 0 && errno != EAGAIN && errno != EINTR) {
                perror("read uart");
                break;
            }
            if (n > 0) {
                derniere_donnee = time(NULL);
                alerte_affichee = 0;

                if (mode == MODE_BRUT || mode == MODE_DEUX)
                    afficher_brut(buf, n);

                if (write(p[1], buf, (size_t)n) != n) {
                    perror("write pipe");
                    break;
                }

                int nb = gps_Read(&gps);
                if (nb < 0) {
                    fprintf(stderr, "gps_Read : erreur\n");
                    break;
                }
                if (nb > 0 && (mode == MODE_TRAITE || mode == MODE_DEUX))
                    afficher_traite(&gps, nb);
            }
        }

        /* ---- Rien ne vient du GPS ? ---- */
        if (!alerte_affichee && time(NULL) - derniere_donnee >= SILENCE_ALERTE_S) {
            printf("!! Aucune donnee depuis %d s : verifier le cablage (TX GPS -> GPIO15), "
                   "l'alimentation, enable_uart=1 et la console serie.\n", SILENCE_ALERTE_S);
            fflush(stdout);
            alerte_affichee = 1;
        }
    }

fin:
    printf("Fin du test.\n");
    close(p[0]);
    close(p[1]);
    UART_Close(uart);
    return 0;
}
```